Experiment 8

Title: Expression Tree Construction, Non-Recursive Post-Order Traversal, and Deletion

Objectives:

- To construct an expression tree from a given prefix expression.
- To implement a non-recursive post-order traversal algorithm for the constructed expression tree.
- To implement a function for the complete deletion of the expression tree.

Problem Statement:
Given a prefix expression (e.g., `+--a*bc/def`), construct its corresponding expression tree. Subsequently, perform a non-recursive post-order traversal of this tree, and finally, delete the entire tree to free up allocated memory.

Outcomes:

- A correctly constructed expression tree representing the given prefix expression.
- Successful non-recursive post-order traversal, printing the expression in postfix form.
- Complete and efficient deletion of the expression tree, preventing memory leaks.

Software and Hardware Requirements:
Software: C++ compiler (e.g., GCC), Integrated Development Environment (IDE) like VS Code or Code::Blocks.
Hardware: Standard personal computer with sufficient RAM and processing power for compilation and execution.

Date of Completion: [Insert Date]

Assessment grade/marks and assessor's sign: [Grade/Marks] / [Assessor's Signature]

Theory Concept in brief:
An expression tree is a binary tree used to represent arithmetic or logical expressions. Internal nodes are operators, and leaf nodes are operands. Prefix expressions (Polish notation) place operators before their operands. Post-order traversal visits the left subtree, then the right subtree, then the root. Non-recursive traversal typically utilizes a stack. Tree deletion involves traversing the tree and freeing memory for each node.

Algorithm: Construct Expression Tree.

a. Read the prefix expression from right to left.
b. Use a stack to store nodes.
c. If a character is an operand, create a new node and push it onto the stack.
d. If a character is an operator, create a new node. Pop two nodes from the stack (right

child then left child) and assign them as children of the operator node. Push the operator node onto the stack.
e. The final node on the stack is the root of the expression tree. Non-Recursive Post-Order Traversal.

a. Use two stacks: `s1` and `s2`.
b. Push the root onto `s1`.
c. While `s1` is not empty:
i. Pop a node from `s1` and push it onto `s2`.
ii. If the popped node has a left child, push it onto `s1`.
iii. If the popped node has a right child, push it onto `s1`.
d. While `s2` is not empty, pop and print the node's data. Delete Tree.

a. Implement a recursive function `deleteTree(node)`:
i. If `node` is NULL, return.
ii. Recursively call `deleteTree(node->left)`.
iii. Recursively call `deleteTree(node->right)`.
iv. Delete `node`.

Flowchart:

Test Cases:

Prefix Expression	Expected Post-order Traversal
`+--a*bc/def`	`abc*de/f--+`
`*+ab-cd`	`ab+cd-*`
`Abc`	`abc`

Test Data Set (if applicable):
Not applicable for this problem, as the input is a single prefix expression.

Mathematical Model (if applicable):
Not applicable. The problem focuses on data structures and algorithms.

Conclusion/Analysis:

The construction of the expression tree from a prefix expression demonstrates the utility of stacks in parsing and tree building. The non-recursive post-order traversal highlights an alternative to recursive approaches, also leveraging stack data structures. The tree deletion ensures proper memory management, preventing resource leaks. The chosen algorithms effectively address the problem, providing a clear and functional solution for expression tree manipulation.